Université Gustave Eiffel

L2 Informatique — Semestre 2

Algorithmique des Arbres

DM 1 — Vérification du statut « de recherche » pour un arbre binaire

---

Membres du groupe :

Raphaël **LIBÉRALE**

Néo **RISTIC**

Groupe de TD : **5**

29 mars 2026

0 Table des matières

1 Réalisation du DM et difficultés rencontrées **2**
1.1 Réalisation et organisation du travail 2
1.2 Difficultés rencontrées 2

2 Protocole expérimental **3**
2.1 Environnement d'exécution 3
2.2 Types d'arbres générés 3
2.3 Paramètres des mesures 4
2.4 Format des résultats 4
2.5 Script de compilation et d'exécution 4

3 Analyse des résultats **4**
3.1 Courbes obtenues 4
3.2 Analyse par méthode 5
3.2.1 Méthode naïve (`est_abr_naif`) 5
3.2.2 Méthode optimisée (`est_abr_definition`) 5
3.2.3 Méthode par parcours infixe (`est_abr_infixe`) 5
3.3 Comparaison des trois méthodes 6
3.3.1 Cas des arbres qui sont des ABR 6
3.3.2 Cas des arbres qui ne sont pas des ABR 6
3.3.3 Synthèse 6

Conclusion **6**

1 Réalisation du DM et difficultés rencontrées

1.1 Réalisation et organisation du travail

L'organisation du travail a été faite via un dépôt Github, lequel nous a permis de travailler en parallèle sur les différentes tâches du devoir et de synchroniser directement les avancées qu'on faisait. Quant au débuggage, l'outil `gdb` a été utilisé ; Cela a ainsi permis de découvrir des méthodes avancées de débuggage comme l'automatisation des breakpoints et des affichages pour aller plus vite à la source du problème.

La répartition du travail est la suivante :

Membre	Parties traitées
Raphaël LIBÉRALE	— Section 2.1.1 — Processus de construction des arbres sur un codage générique (`construit_quelconque`) — Section 2.1.2 — Génération des parcours préfixes des arbres complets, filiformes et quelconques. — Section 2.1.3 — Génération d'arbres ABR et non-ABR complets, filiformes et quelconques
Néo RISTIC	— Section 1.1 — Méthode naïve (`est_abr_naif`) — Section 1.2 — Méthode optimisée (`est_abr_definition`) — Section 1.3 — Méthode infixe (`est_abr_infixe`) — Section 2.2 — Protocole expérimental et script Python (`analyse_complexite.py`)

Table 1 – Répartition des tâches au sein du binôme.

1.2 Difficultés rencontrées

Raphaël LIBÉRALE : La génération des parcours préfixes depuis les parcours infixes a été technique car il fallait trouver le bon algorithme pour chaque forme d'arbre. Plus particulièrement, hormis le débuggage avec `gdb` assez long avec chacune des fonctions, la création du parcours préfixe d'un arbre filiforme a pris plusieurs essais d'algorithmes, dont les premiers étaient infructueux.

Ensuite, lors des générations d'arbres en fonction des formes, il fallait trouver comment implémenter les caractères de fin pour le codage de l'arbre (en l'occurrence `-1`) ; J'ai choisi

de le faire avant la création du parcours infixe sur les arbres complets (donc le tableau `infixe` est de taille $2n + 1$ avant l'appel de la fonction de parcours préfixe, où n est le nombre de nœuds). Pour les arbres filiformes et quelconques, cela se fait lors de la création du parcours infixe (donc `infixe` est de taille n avant l'appel de la fonction).

De plus, les tests étant automatisés, j'ai donc dû trouver un substitut des fonctions `pow`, `log2` et `floor` de la bibliothèque `<math.h>`, car je devais me passer de l'option de compilation `-lm`. J'ai donc trouvé sur internet une manière de faire en procédant avec les opérations bit à bit implémentées en `C` afin de reproduire ces calculs

Néo RISTIC : La première difficulté que j'ai rencontré a été la création de la fonction `est_abr_infixe` ou plus particulièrement sa fonction auxiliaire. Cette dernière n'est pas nécessairement longue mais m'a demandé une réelle compréhension récursive de l'algorithme. J'ai donc pris un crayon et une feuille et essayé de comprendre en dessinant des parcours infixe d'arbres binaires.

Mais le plus long a été la partie d'analyse des complexité. Notamment car les résultats graphiques obtenus paraissaient incorrects, ou du moins non représentatifs des complexités. Au final, nous avons tous les deux résolu le problème ... (pas encore mdr)

2 Protocole expérimental

Cette section décrit le protocole mis en place pour comparer expérimentalement les trois méthodes de vérification du statut ABR. L'objectif est qu'un lecteur puisse reproduire intégralement nos expériences.

2.1 Environnement d'exécution

- Système d'exploitation : Ubuntu 24.04.4
- Compilateur : `clang`, version 18.1.3
- Processeur : Intel(R) Core(TM) Ultra 7 155H
- Mémoire vive : 30 Gio de RAM (swap : 8 Gio)

2.2 Types d'arbres générés

Nous avons généré six types d'arbres binaires sans doublon, résumés dans le tableau ci-dessous.

Morphologie	Type	Fonction de génération
Presque complet	ABR	`ABR_presque_complet_alea`
Presque complet	Non-ABR	`non_ABR_presque_complet_alea`
Filiforme	ABR	`ABR_filiforme_alea`
Filiforme	Non-ABR	`non_ABR_filiforme_alea`
Quelconque	ABR	`ABR_quelconque_alea`
Quelconque	Non-ABR	`non_ABR_quelconque_alea`

TABLE 2 – Types d'arbres utilisés dans les expériences.

2.3 Paramètres des mesures

- Tailles testées : $n \in \{100, 1100, 2100, 3100, 4100, 5100, 6100, 7100, 8100, 9100\}$ nœuds
- Nombre de répétitions par point de mesure : 50
- Métrique mesurée : nombre de nœuds visités (type `long long`)

2.4 Format des résultats

Les résultats sont stockés dans le fichier `mesures.csv` dont le format de la première ligne est :

`Taille;Morphologie;Methode;Nb_visites;Temps`

2.5 Script de compilation et d'exécution

Grâce à la création d'un `makefile`, la compilation est hautement simplifiée et cela permet de reproduire les expériences suivantes avec les simples commandes suivantes :

```
make
./programme
python3 analyse_complexite.py
```

Listing 1 – Compilation et exécution

3 Analyse des résultats

3.1 Courbes obtenues

FIGURE 1 – Nombre de nœuds visités en fonction de la taille — arbres ABR.

FIGURE 2 – Nombre de nœuds visités en fonction de la taille — arbres non-ABR.

3.2 Analyse par méthode

3.2.1 Méthode naïve (`est_abr_naif`)

Complexité théorique. Pour chaque nœud n, la méthode naïve appelle `abr_max` sur le sous-arbre gauche et `abr_min` sur le sous-arbre droit, ce qui implique de reparcourir les sous-arbres. En notant N le nombre de nœuds, la complexité dans le pire cas est donc ...

Résultat clé

Complexité temporelle de `est_abr_naif` : $\mathcal{O}(n^2)$ dans le pire cas.

Validation expérimentale. Le pire cas est atteint premièrement sur un arbre de type filiforme car sa hauteur h est égale à $n-1$, où on a n le nombre de nœuds de l'arbre. Deuxièmement, pour avoir une complexité dans le pire cas, il faut avoir un arbre où tous les enfants sont des fils gauches, ou sont des fils droits ; Dans le cas d'un arbre à 1000 nœuds (qui est la deuxième plus petite taille testée sur notre échantillon total), on a une probabilité d'avoir un arbre de cette forme-ci qui vaut $\frac{1}{2^h}$., soit dans notre cas $\frac{1}{2^{999}} \approx 0$.

On remarque sur les graphiques qu'en pratique, on atteint une complexité plus proche de $\theta(4n)$; En effet, on a $n_1 + n_2 + 1 + n_3 + n_4$ nœuds concrets

Différences selon la morphologie. ...

3.2.2 Méthode optimisée (`est_abr_definition`)

Complexité théorique. La méthode optimisée propage les valeurs min/max lors d'un unique parcours, visitant chaque nœud au plus une fois. Sa complexité est donc ...

Résultat clé

Complexité temporelle de `est_abr_definition` : $\mathcal{O}(\ldots)$ dans le pire cas.

Validation expérimentale. ...

Différences selon la morphologie. ...

3.2.3 Méthode par parcours infixe (`est_abr_infixe`)

Complexité théorique. La méthode infixe réalise un parcours infixe de l'arbre en s'interrompant dès la première violation. Dans le meilleur cas (violation proche de la racine), sa complexité est $\mathcal{O}(\ldots)$; dans le pire cas (arbre ABR), elle est $\mathcal{O}(\ldots)$.

Résultat clé

Complexité temporelle de `est_abr_infixe` : $\mathcal{O}(\ldots)$ dans le pire cas.

Validation expérimentale. ...

Différences selon la morphologie. ...

3.3 Comparaison des trois méthodes

3.3.1 Cas des arbres qui sont des ABR

...

3.3.2 Cas des arbres qui ne sont pas des ABR

...

3.3.3 Synthèse

	Naïve	**Optimisée**	**Infixe**
Complexité pire cas	$\mathcal{O}(?)$	$\mathcal{O}(?)$	$\mathcal{O}(?)$
Complexité meilleur cas	$\mathcal{O}(?)$	$\mathcal{O}(?)$	$\mathcal{O}(?)$
Avantage principal	...	...	...
Inconvénient principal	...	...	...

Table 3 – Synthèse comparative des trois méthodes.

3 Conclusion

...